MIS 3371 · TRANSACTION PROCESSING SYSTEMS I · SECTION 20300

Task: Build PayDesk

A modern transaction web app, built in two layers - CSS for how it looks, JavaScript for what it does

What you are building

A single-page payment desk. Someone picks an account, types who they are paying and how much, and the app decides on the spot whether the payment posts, gets held, or is declined - then shows a receipt and adds it to a running ledger.

There is no page reload anywhere in it, and no server. Three files: index.html for the structure (given to you), style.css for the look, app.js for the behaviour and the decisions.

The whole point of the task is the line between those last two files. If you ever write an if statement in the CSS, or a colour in the JavaScript, you have crossed it.

The two layers, and what belongs in each

	CSS does	JavaScript does
Fields	Size, spacing, border, focus ring	Reads the values out of them
Errors	What "wrong" looks like - red border, red text	Decides WHICH fields are wrong, and writes the message
Status	What POSTED, HELD and DECLINED look like	Decides which one this payment is
The button	Its colour, and what the busy state looks like	Disables it, and runs the payment when it is clicked
The ledger	Table layout, alignment, row lines	Builds the rows from the transactions array
Small screens	Collapses to one column	Nothing - this is CSS's job alone
The totals	How the three tiles are laid out	Loops the array and counts them

The test for which file something belongs in

Could a blind user with a screen reader still get the right answer with your CSS switched off? If not, a decision has leaked into the CSS layer.

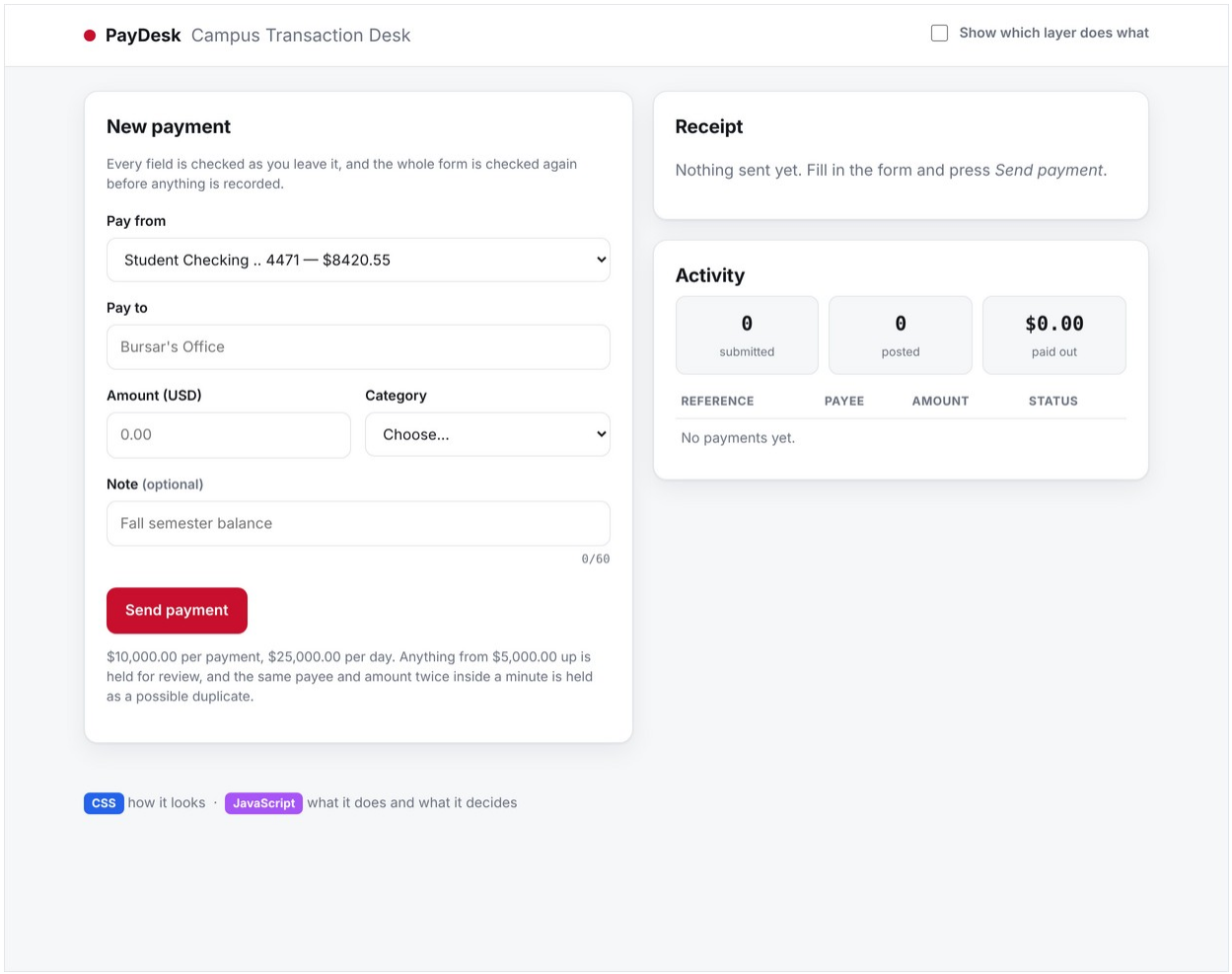
Does a function in app.js mention a colour, a pixel, or a font? Then a presentation detail has leaked into the JavaScript layer.

Both leaks cost marks, and both are easy to avoid if you check as you go rather than at the end.

What the finished app looks like

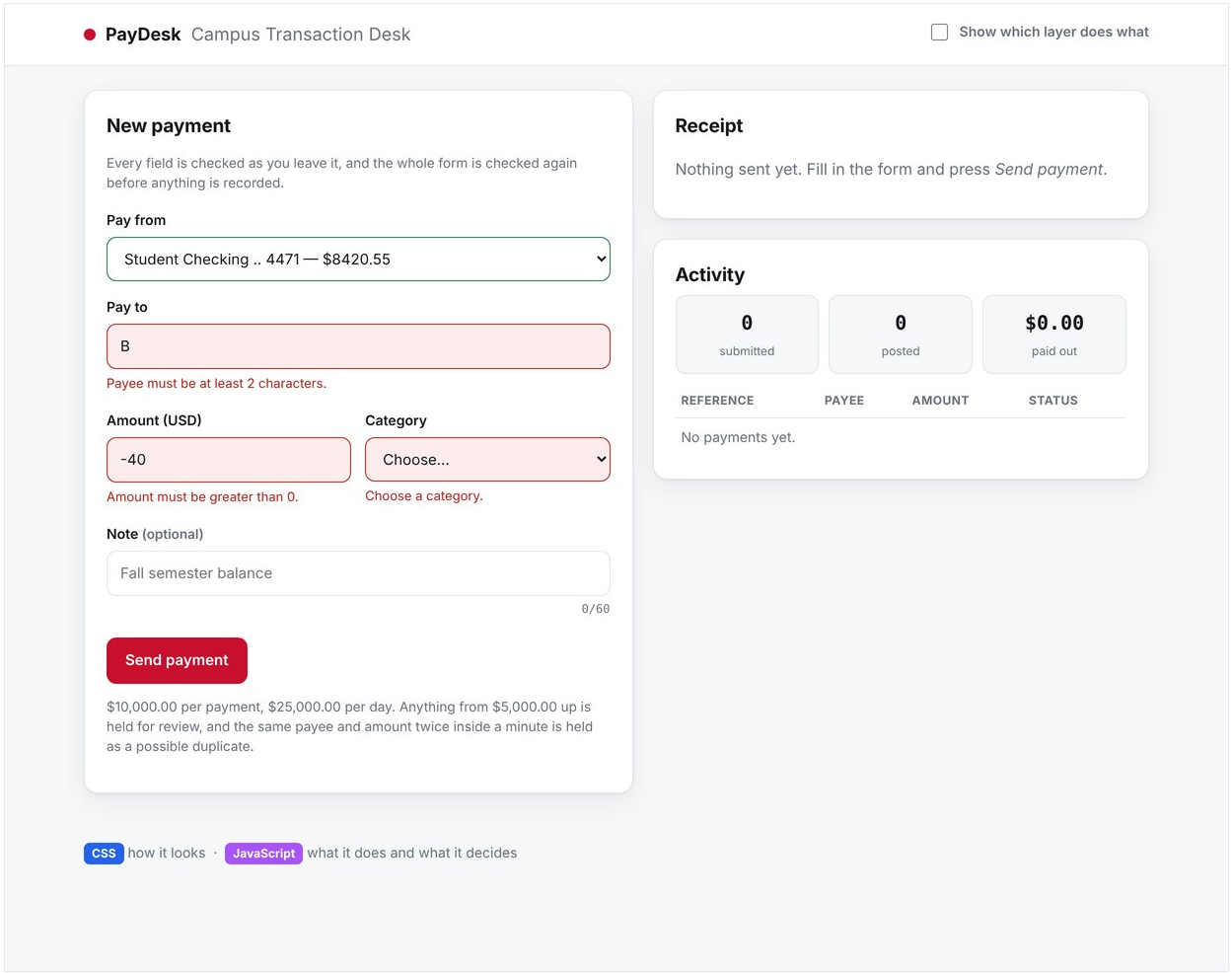
These are screenshots of the working reference version. Yours does not have to match pixel for pixel - the colours and the spacing are your decisions - but every state shown here has to work.

1. The starting state



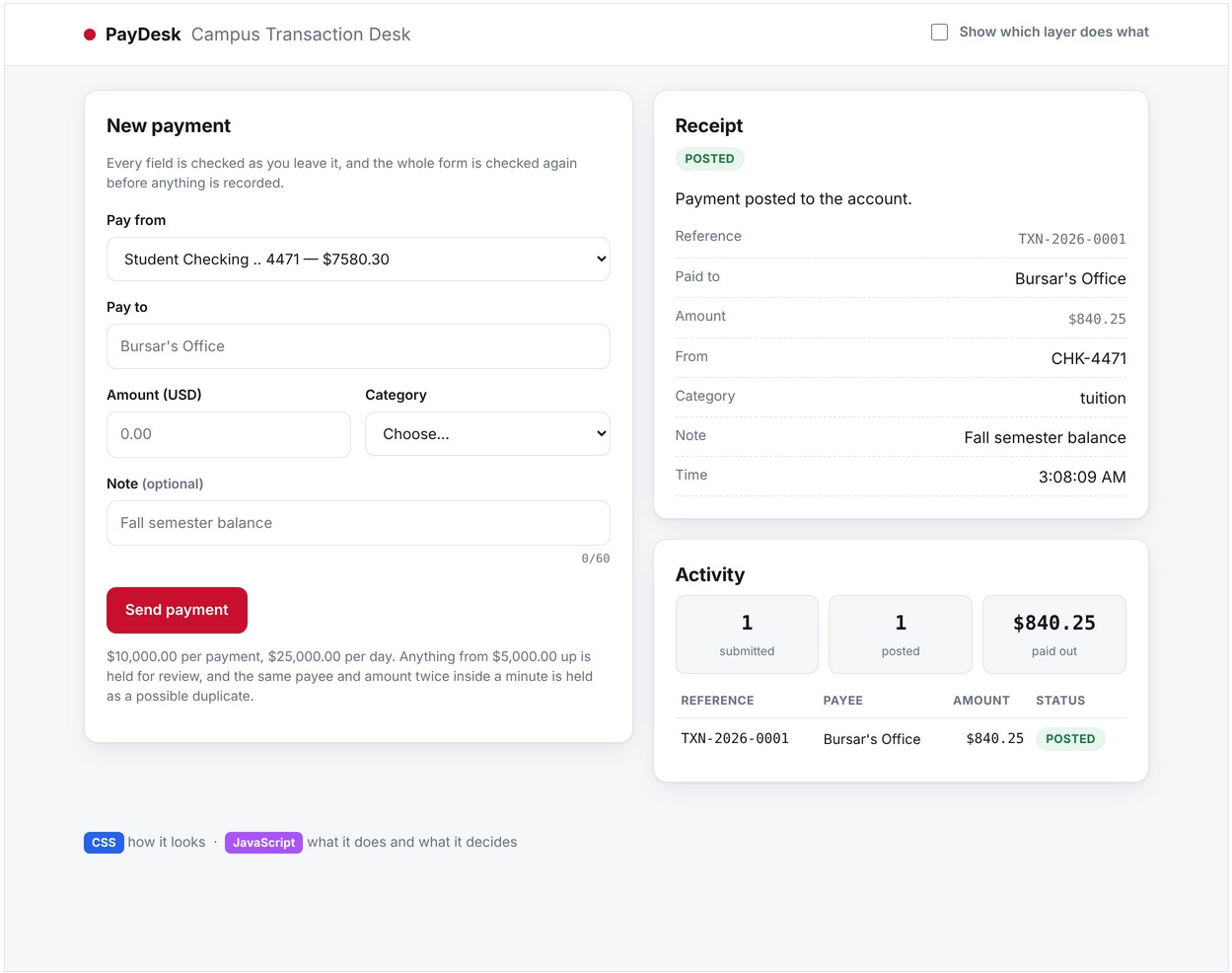
An empty form, an empty ledger, and three accounts with their balances. Nothing has been decided yet.

2. The JavaScript layer catching bad input



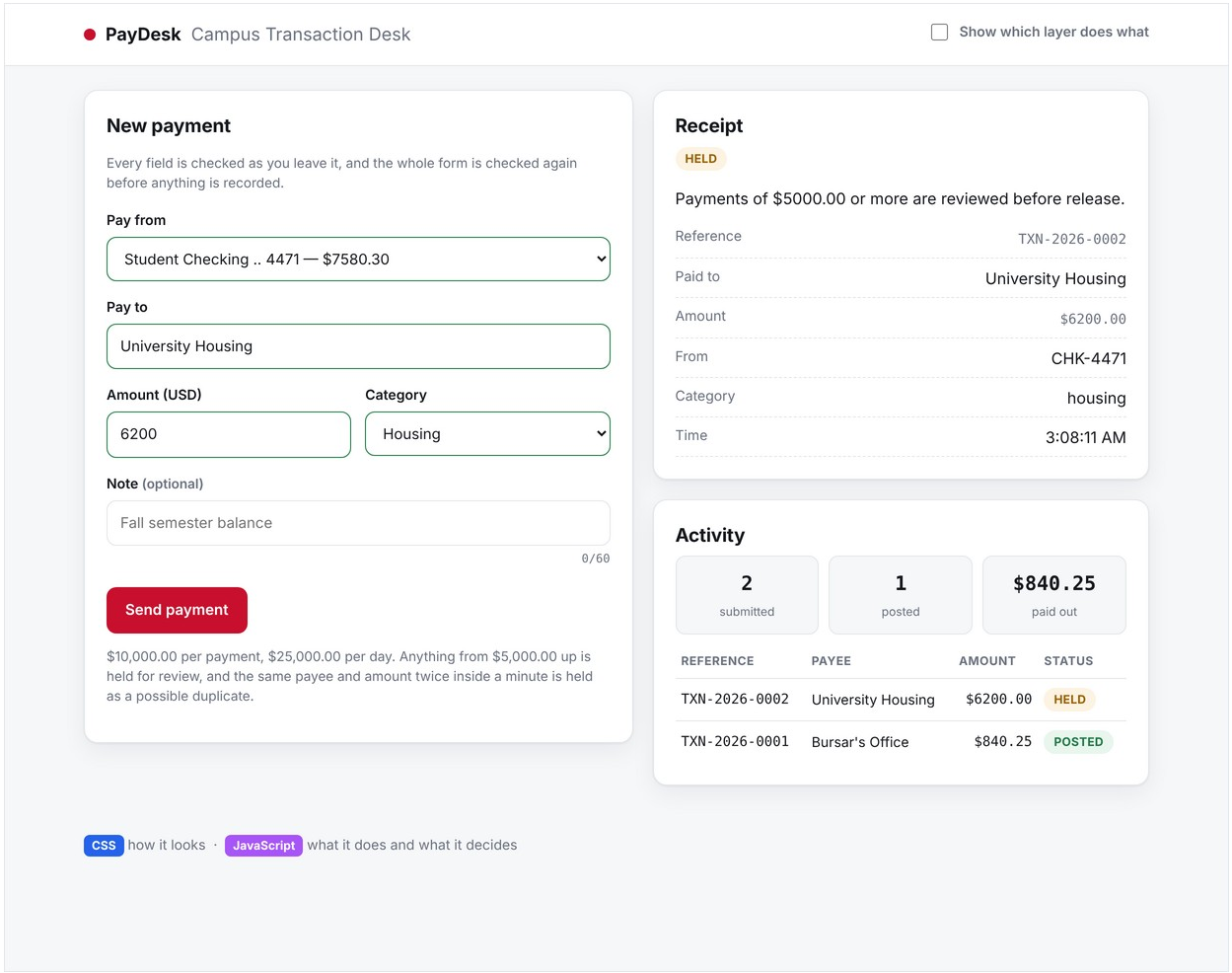
Three fields are wrong, each with its own message. The red borders are CSS; knowing which fields to put them on is JavaScript.

3. A payment that posts



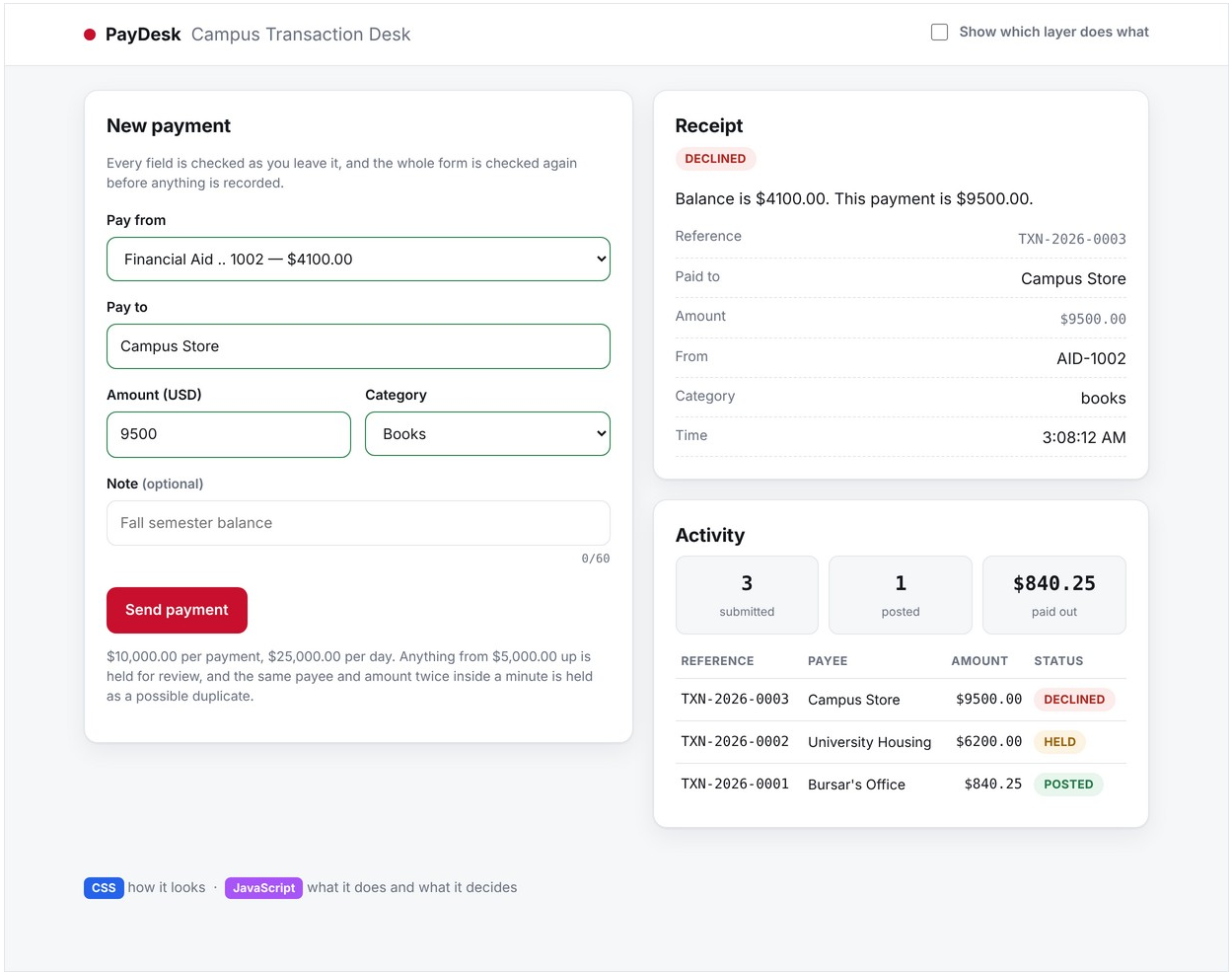
$840.25 to the Bursar's Office. A receipt appears, the balance drops from $8,420.55 to $7,580.30, and the ledger gains a row.

4. A payment that policy holds



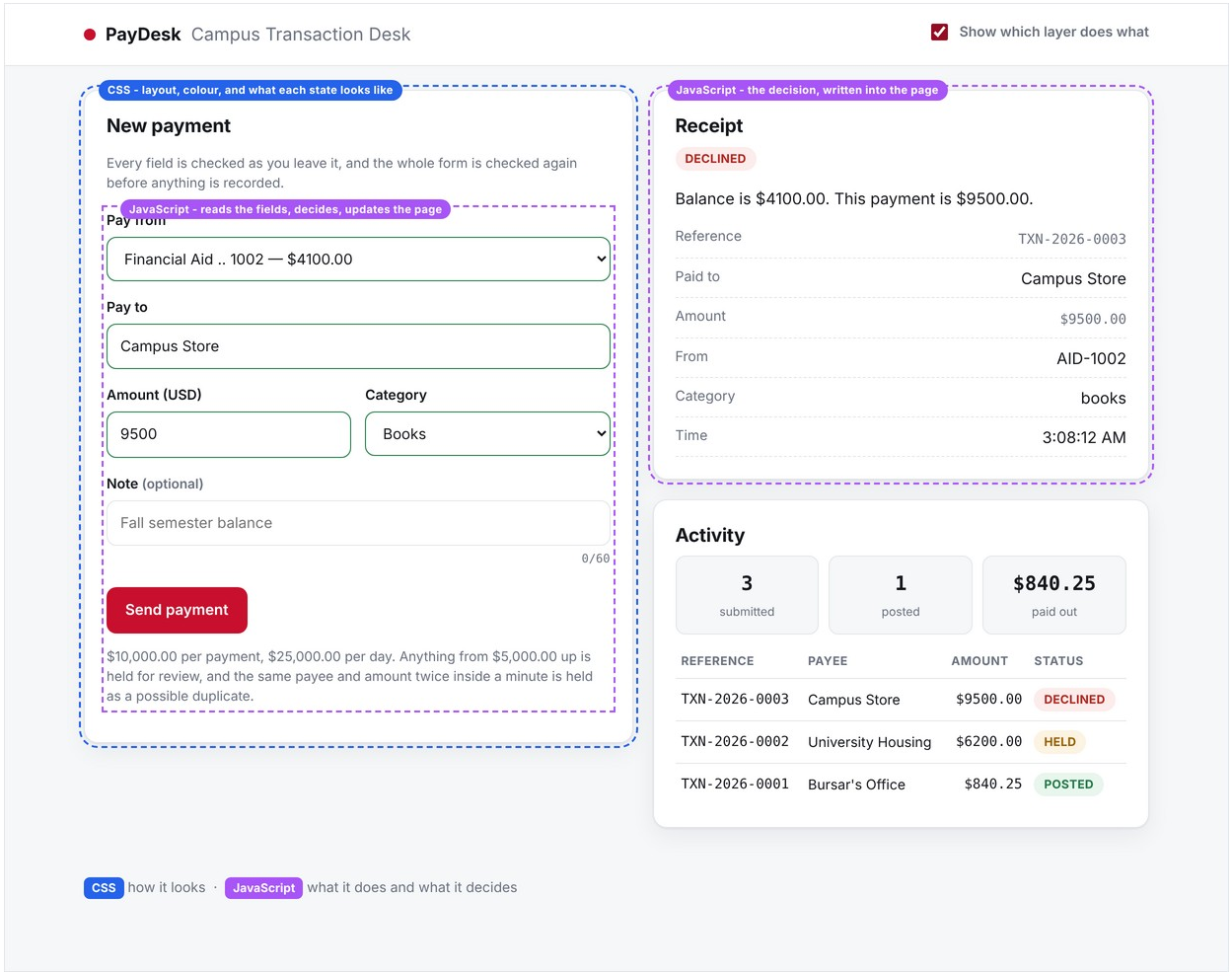
$6,200 is over the review threshold, so it is HELD rather than refused. Held is not an error - telling the two apart is the point.

5. A payment that is declined



$9,500 from an account holding $4,100. The data was perfectly valid; policy is what stopped it.

6. The layer view



The toggle in the header outlines every region and names the layer responsible for it. Build this and you will not get the two mixed up.

What to build

The starter has every one of these marked as a TODO in the file it belongs in. Work down the list; the app comes alive in pieces.

The CSS layer - style.css

#	What to write
C1	Design tokens on :root - background, surface, text, muted text, border, brand, radius
C2	Page basics: margin, background, colour, font family, line-height, box-sizing
C3	The .wrap container - max width, centred, side padding
C4	The two-column layout with CSS grid
C5	The .card surface - background, border, radius, shadow, padding
C6	Form controls - full width, padding, radius, font: inherit, and a visible focus ring
C7	The state classes JavaScript sets: input.invalid, input.valid, and .err
C8	Buttons, and the three status pills: posted, held, declined
C9	Responsive - one column under 900px

The JavaScript layer - app.js

#	What to write
J1	toAmount(text) - text to number, NaN when empty
J2	validate(form) - returns {} or { field: message }
J3	postedToday(code) - total posted from this account since midnight
J4	recentDuplicate(code, payee, amount) - same payment inside a minute
J5	applyRules(form) - returns { status, reason, message }
J6	processPayment(form) - validate, apply rules, record, update the balance
J7	readForm() - the five field values as one object
J8	showErrors(errors) - set the messages and the class names
J9	renderAccounts() - fill the account list with live balances
J10	renderReceipt(txn) - write the decision into the page
J11	renderLedger() - the table rows and the three totals
J12	Wire up the events: blur, input, and submit

The rules your app must enforce

Rule	Outcome	Why it cannot be a CSS rule
Amount is empty, zero, negative, or not a number	Rejected before anything else	It is a question about a value, not about appearance
Payee missing, or over 40 characters	Rejected	Same
Amount over $10,000	Rejected	A limit is a number comparison

Rule	Outcome	Why it cannot be a CSS rule
Amount over the account balance	DECLINED	Needs the balance, which only the data knows
Account is over $25,000 for the day	DECLINED	Needs every earlier payment today
Same payee and amount inside 60 seconds	HELD	Needs to look at history
Amount of $5,000 or more	HELD for review	A threshold, decided in code
Everything else	POSTED	

Order matters, and you will be asked about it

A $9,500 payment from an account with $4,100, submitted twice in ten seconds, trips three rules at once. Which message should the user see?

There is no single right answer, but there is a right way to decide: pick the one that is most useful to the person reading it, and write one sentence in a comment saying why you put that branch first.

If you finish early

	What to add
S1 Dark mode	Redefine the tokens inside @media (prefers-color-scheme: dark). If you built C1 properly this is about eight lines and nothing else changes.
S2 Pending state	Show a spinner on the button for a moment before the receipt appears, so the app feels like it is working.
S3 Layer view	The toggle from screenshot 6: outline every [data-layer] element and show its note. Pure CSS plus one class name from JavaScript.
S4 Touched fields	Only complain about a field after the user has left it once. Much less annoying to use, and only a few lines.
S5 Filter the ledger	A row of buttons that shows only posted, only held, or all - without reloading or losing data.
S6 Keyboard only	Do a whole payment without touching the mouse. Fix whatever you cannot reach.

PayDesk Campus Transaction Desk

Show which layer does what

New payment

Every field is checked as you leave it, and the whole form is checked again before anything is recorded.

Pay from

Financial Aid .. 1002 — $4100.00

Pay to

Campus Store

Amount (USD)

9500

Category

Books

Note (optional)

Fall semester balance

0/60

Send payment

$10,000.00 per payment, $25,000.00 per day. Anything from $5,000.00 up is held for review, and the same payee and amount twice inside a minute is held as a possible duplicate.

Receipt

DECLINED

Balance is $4100.00. This payment is $9500.00.

Reference	TXN-2026-0003
Paid to	Campus Store
Amount	$9500.00
From	AID-1002
Category	books
Time	3:08:12 AM

Activity

3	1	$840.25
submitted	posted	paid out

REFERENCE	PAYEE	AMOUNT	STATUS
TXN-2026-0003	Campus Store	$9500.00	DECLINED
TXN-2026-0002	University Housing	$6200.00	HELD
TXN-2026-0001	Bursar's Office	$840.25	POSTED

CSS how it looks · JavaScript what it does and what it decides

S1 and the responsive layout together: same HTML, same JavaScript, different tokens and one media query.

How it is marked

	Points	Notes
The app runs with no console errors	10	Open the console before you submit
CSS layer C1-C9 complete	25	Readable, token-driven, no magic colours repeated everywhere
JavaScript layer J1-J12 complete	35	Correct outcomes for every rule in the table
The two layers stay separate	15	No decisions in CSS, no styling in JavaScript
Responsive and usable by keyboard	10	Narrow the window; tab through the form
Comments that say why, not what	5	Especially on the order of your rules
Total	**100**	

What to hand in

- The three files, in a folder, in your repository.
- Screenshots of your own app in all four states: empty, a validation error, a posted payment, and a declined one.
- Two or three sentences on the order of your rules in J5 - which check you put first and why.
- A commit message that says what you did, not "update".

Be ready to explain any line of it

The syllabus says code you cannot explain may receive no credit, and the individual code defence is worth 10% of the course.

Pick any function in app.js. Be able to say out loud what goes in, what comes out, and why the lines are in that order. If a tool wrote a line you cannot explain, delete it and write one you can.